

锦标赛

先假设最终胜利的是 i 号选手，那么我们有 $1 - i$ 的胜者是 i ，同理 $i - n$ 的胜者也是 i 。

考虑设计四个状态 $gl_{l,r}, gr_{l,r}, fl_{l,r}, fr_{l,r}$

其中四个状态分别表示： $[l, r]$ 作为独立的游戏，那么最后一步只剩下 l, r 且 l 最后获胜，其中游戏的方案数乘以对应的概率； $[l, r]$ 作为独立的游戏，那么最后一步只剩下 l, r 且 r 最后获胜，其中游戏的方案数乘以对应的概率； $[l, r]$ 作为独立的游戏， l 最后获胜，其中游戏的方案数乘以对应的概率； $[l, r]$ 作为独立的游戏， r 最后获胜，其中游戏的方案数乘以对应的概率

注意到 gl/gr 的转移可以枚举 l 和 r 之间的唯一分界线 k ，其中 $[l, k]$ 胜者为 l ， $[k + 1, r]$ 胜者为 r ，那么这两者都可以从 fl/fr 转移而来。

对于 fl/fr 的转移任然可以考虑枚举最后一步中被删去的人，然后从 $gl/gr/fl/fr$ 转移而来。

最后求值只需根据 $fr_{1,i}$ 和 $fl_{i,n}$ 的值计算即可。

针对部分分， $O(n^5)$ 可以设计状态 $f_{i,j,k}$ 表示 k 是 $[i, j]$ 的胜者，而事实上注意到这个我认为就能比较容易注意到正解。

胡言乱语

考虑记录 $f_{a,b}$ 表示对应询问的答案。我们尝试是否能递推。

如果利用常规的组合数递推方式将涉及到 n 的变化，那么我们可以尝试使用等式

$$\binom{i}{a} = \binom{i-1}{a-1} + \binom{i-1}{a}, \binom{n-i}{b} = \binom{n-i+1}{b} - \binom{n-i}{b-1}$$

那么对于 $a, b > 0$ 的情况，有：

$$\begin{aligned} & \sum \binom{i}{a} \binom{n-i}{b} x^i y^{n-i} \\ &= \sum \left(\binom{i-1}{a-1} + \binom{i-1}{a} \right) \left(\binom{n-i+1}{b} - \binom{n-i}{b-1} \right) x^i y^{n-i} \\ &= \sum \binom{i-1}{a-1} \binom{n-i+1}{b} x^i y^{n-i} + \sum \binom{i-1}{a} \binom{n-i+1}{b} x^i y^{n-i} - \sum \binom{i}{a} \binom{n-i}{b-1} x^i y^{n-i} \\ & f_{a,b} = \frac{1}{y} f_{a-1,b} - f_{a,b-1} + \frac{x}{y} f_{a,b} \end{aligned}$$

那么我们只需要分别考虑 $a = 0/b = 0$ 或 $x = y/x = 0/y = 0$ 即可。

a=0/b=0

考虑这是一个组合数带 $\frac{x}{y}$ 的列求和，是可以递推的

x=y

这相当于要求 $\sum \binom{i}{a} \binom{n-i}{b}$

那么我们只需要考虑可以组合意义对应到从 $n+1$ 个球当中选取 $a+b+1$ 个，并抽取第 $a+1$ 个，因此答案是 $\binom{n+1}{a+b+1}$

最终复杂度 $O(m^2)$ ，其中 m 是 a, b 的上界。

树上分糖果·题解

算法1

考虑枚举袋子容量上限，暴力计算查询时袋子里的糖果数量。但是袋子容量上限非常大，不一定能通过什么数据。

期望得分 0 分。

算法2

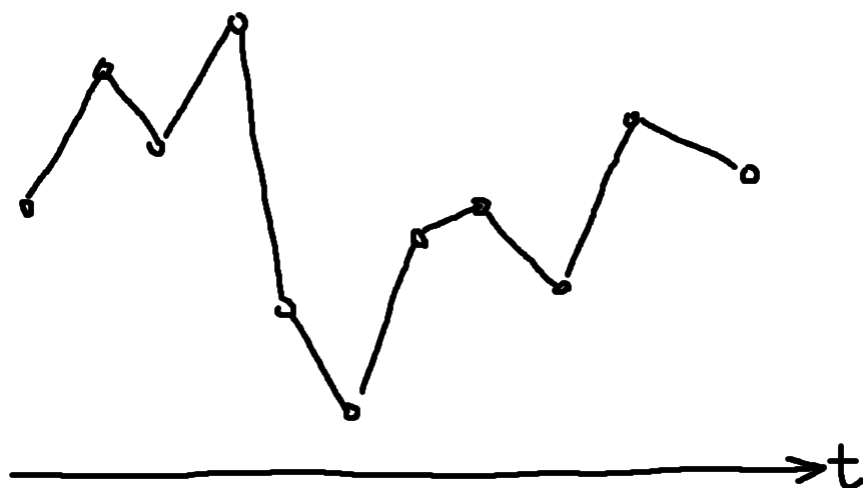
不难想到一个性质：袋子容量越大，最后的糖果就越多。即在操作过程不变的情况下，如果一开始换成更大的袋子，则最后的糖果数量不会更少。

容量与剩余这两者呈单调不降的关系，因此答案只有三种情况：有界区间、无上界区间、无解。

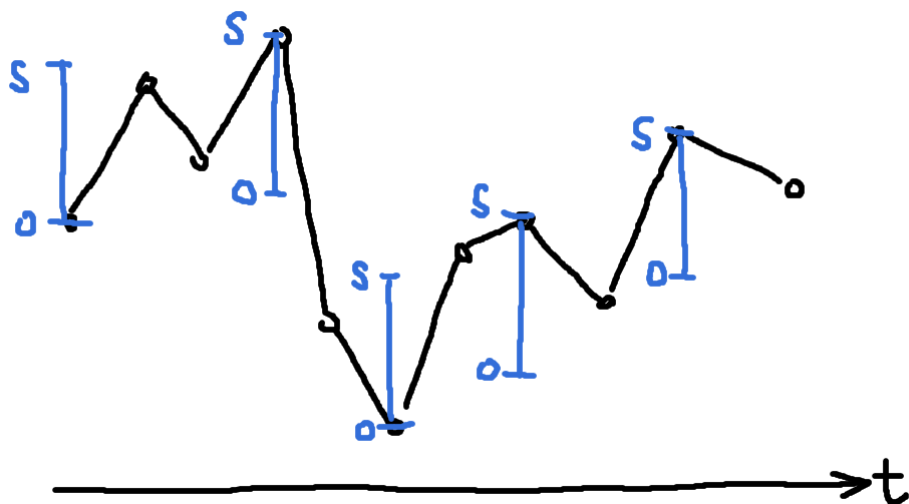
考虑二分答案的方法去计算区间的上下界，验证时需要计算剩余糖果数。如果你只会暴力那就很难拿到较高的分数。

于是将每次放入和取出糖果的信息建一棵时间轴的线段树，在线段树上查询剩余糖果数。

如果允许袋子中的糖果数量无上限且也可以是负数，则糖果数量可以视为一条折线，例如下图：

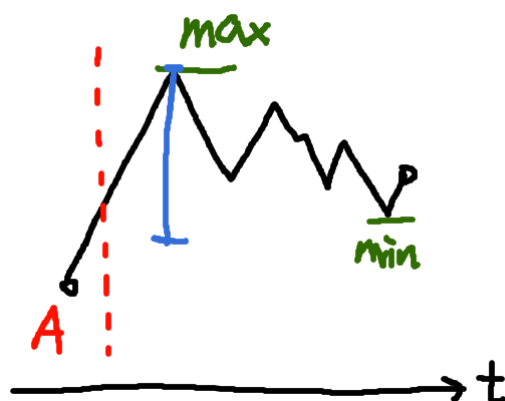


然后再加上袋子的糖果容量限制，要求糖果数量始终在 $[0, s]$ 范围内，则相当于将纵坐标轴上下拖动，如下图



要算出最终的剩余糖果数量，只需找出最后一次拖动纵坐标轴是什么时候。

加入最后一次拖动是向上拖动，则这次拖拽之后的折线满足 $\max - \min \leq s$ 。如下图所示。我们可在线段树上二分出A点：A点不算自身的右侧这段满足 $\max - \min \leq s$ 。再检查A点是 $< \min$ 或者是 $> \max$ ，以此来判断最后一次拖动是向上拖动到 $\max$ 还是向下拖动到 $\min$ ，从而算出最终的剩余糖果数量。



二分答案需要 $O(\log s)$ 次，每次验证需要在线段树上 $O(\log q)$ 查询，即每次查询复杂度 $O(\log s \log q)$ 。

那么如何拿出每个节点的线段树呢？

对于树是一条链的情况，每次修改相当于一个区间。离线，依次枚举链上节点，维护一棵线段树，一次修改操作会被加入线段树 1 次和从线段树中删除 1 次。

对于一般的树，还是离线，用树上差分常见操作将一次修改的链拆分为 4 条节点到根的链，考虑在树上自下而上的处理所有询问，可以将所有儿子线段树合并然后再执行当前节点到根的修改，以得到当前节点的线段树。

线段树具体实现，可以用“区间修改，查询区间最大最小值”的线段树，也可以使用“单点修改，查询前缀和的最大最小值”的线段树。我两种都试了试，后者稍微快一点点（快10%左右），原因主要是单点修改线段树的节点数量会少一些。

线段树部分时间复杂度 $O((n + q) \log q)$ 。

假设 $n \approx q$ ，则总时间复杂度 $O(q \log q \log s)$ ，期望得分 85 ~ 100 ，取决于程序实现时的常数。

算法3

上一个算法中，二分答案之后，需要在线段树上二分来验证。这启发我们思考：能不能直接在线段树上计算出袋子容量 s 的上下界？

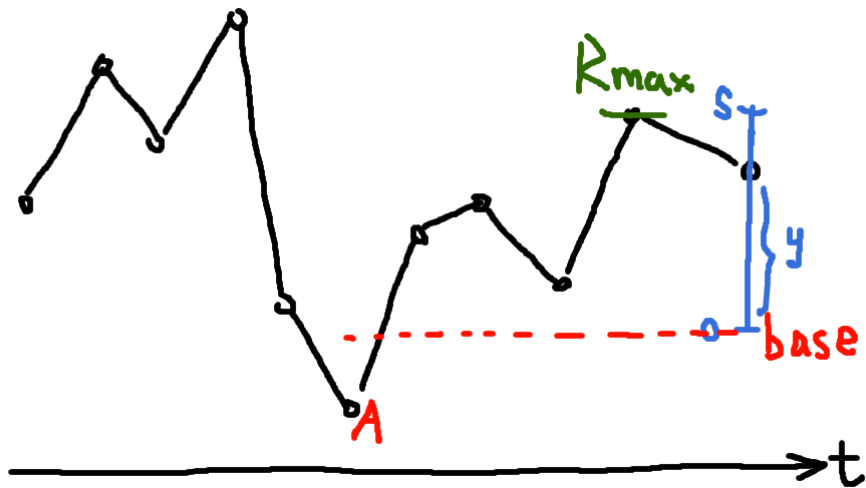
纵坐标轴最终被拖拽到了什么地方，我们是知道的。纵轴原点的位置是查询时刻折线数值减去剩余容量，设为 $base$ 。



考虑去找出算法2中提到的A点：即折线上从右向左，第一个低于 $base$ 的点。

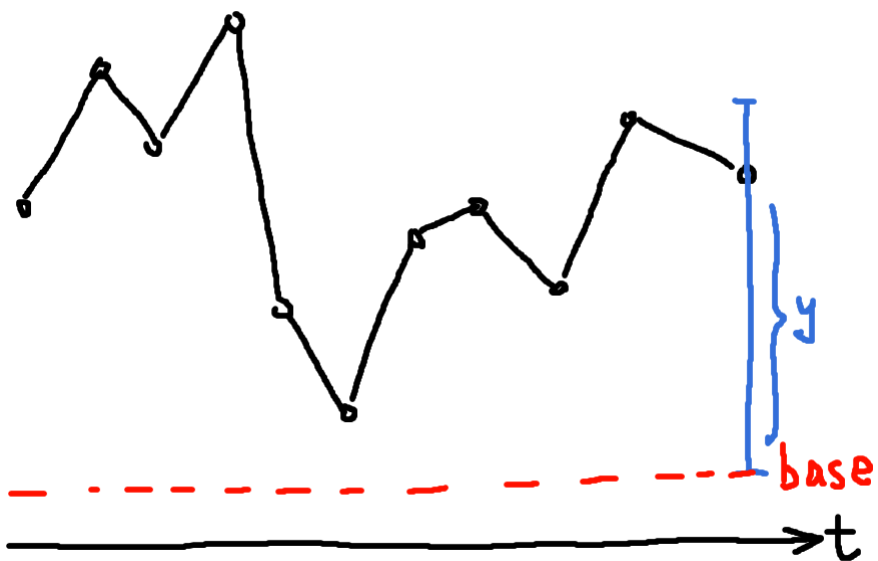
情况1: A点在 $base$ 线以下

这种情况下，坐标轴一定是在A点右侧被向上拖动到目标位置。所以拖动后的 s 位置就是A点右侧的 max 。因此 $s = Rmax - base$ ，是唯一的答案。



情况2: A点不存在

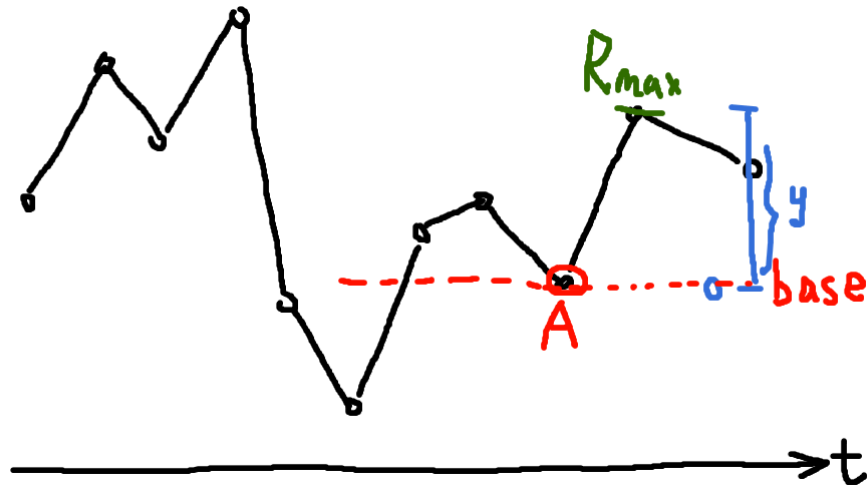
这种情况下，坐标轴不可能被拖动到目标位置，无解。



情况3: A点刚好在 $base$ 线上

这种情况下， $s = Rmax - base$ 一定是一个符合要求的解。

我们还得判断 $s > Rmax - base$ 是否可能也是解。

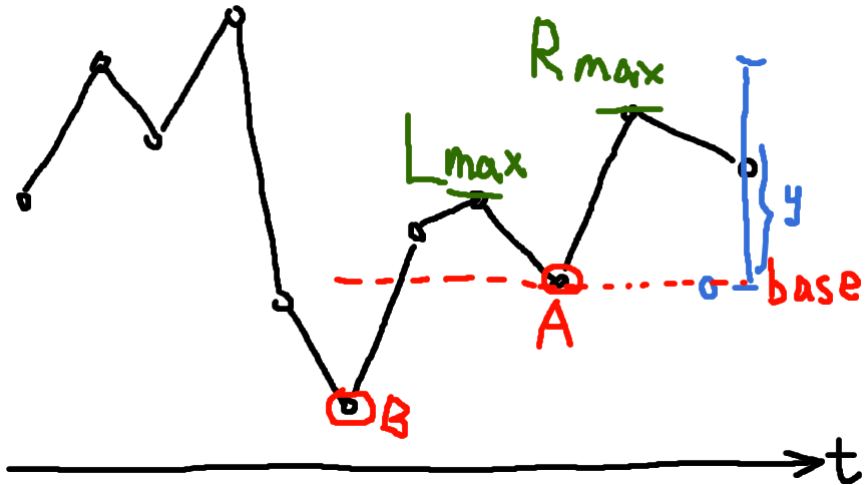


当 $s > Rmax - base$ 时，纵轴必须由A点向下拖动到目标位置。所以A点左侧必须有一个比 $Rmax$ 更高的点。

设B点是从A点向左第一个严格在 $base$ 以下的点， $Lmax$ 是 $[B, A]$ 的最大值， $Rmax$ 是 A 右侧的最大值。

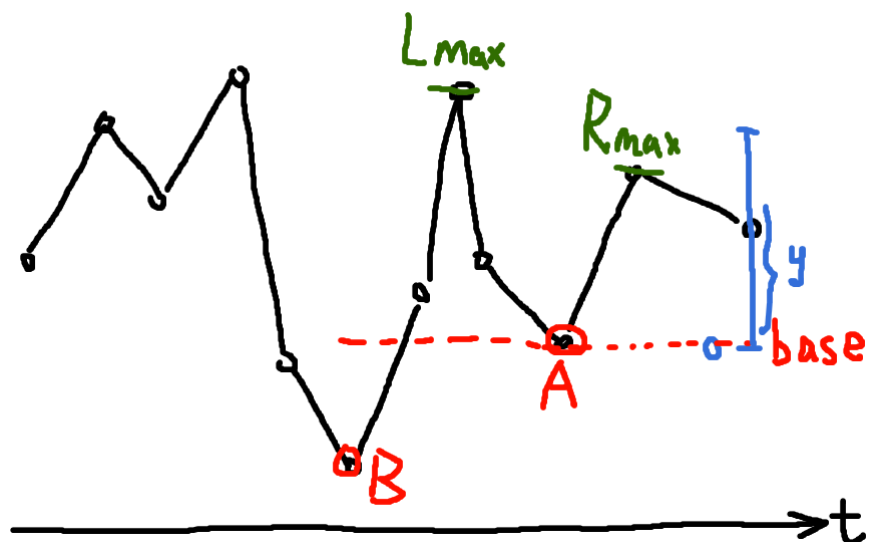
情况3-1: $Lmax \leq Rmax$

这种情况下如果 $s > Rmax - base$ 则纵轴无法拖动到目标位置，所以答案也只可能是 $s = Rmax - base$ 。



情况3-2: $Lmax > Rmax$

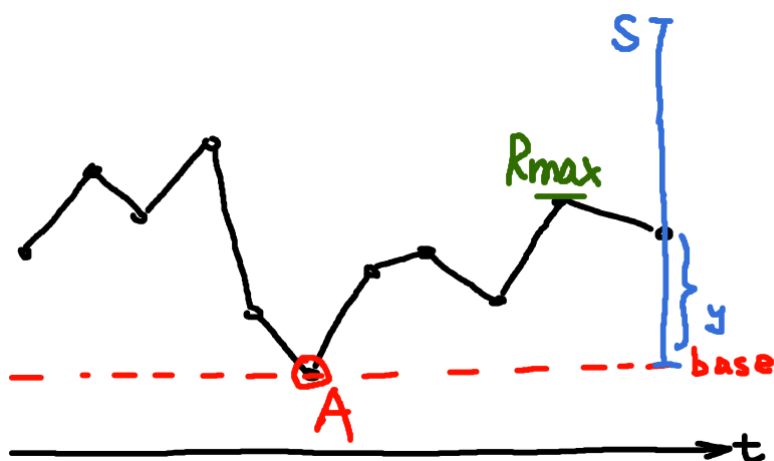
这种情况下， $s \in [Rmax - base, Lmax - base]$ 都是符合要求的。



情况3-3: B点不存在

这种情况下, 只要 $s \in [Rmax - base, +\infty)$, 纵轴就会被A点拖动到目标位置。

不过因为题目限制, 在计算总和时可以认为 $+\infty = 10^{18}$ 。



总结一下以上几种情况, 每种情况所需计算的数值, 都可以在线段树上 $O(\log q)$ 计算出来。因此一次询问操作时间复杂度降低到 $O(\log q)$ 。

总时间复杂度 $O((n + q) \log q)$, 期望得分 100 分。本题由“IOI2021分糖果”的基础上进行了较大的改编而来。